

Rana Omarhan

230103191

HOST CPU / SOC MODEL: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz

OPERATING SYSTEM & KERNEL: Windows 11 Pro (OS Build 22631), Kernel 10.0.22631

Task1

CPU Model & Architecture: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz (Tiger Lake, x86-64)

Base & Max Boost Clock: Base: 2.40 GHz (Task Manager shows 2.42 GHz) / Max Boost: 4.20 GHz

Physical Hardware Cores:4

Logical Cores (SMT/Threads):8

L1 Data / Instruction Cache: L1d: 192 KB (4 × 48 KB) / L1i: 128 KB (4 × 32 KB) / Total L1: 320 KB

L2 Cache (Per Core / Total): Per Core: 1.25 MB / Total: 5.0 MB

L3 Shared Last-Level Cache: 8.0 MB

SIMD Vector Extensions: SSE4.2, AVX2, AVX-512 (verify with CPU-Z or CoreInfo)

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

Flynn's taxonomy isn't really a single label for my whole machine — it depends on what's running at the moment. On my i5-1135G7, I can see all three modes show up in different situations.

SISD happens when I run plain single-threaded code. For example, a normal C loop or a single-threaded Python script. One core is fetching and executing one instruction at a time, working on one piece of data.

SIMD shows up when the CPU uses vector instructions like AVX2 or AVX-512. For example, matrix multiplication or image processing. One instruction can operate on several floats or integers at once, instead of just one.

MIMD is what happens when multiple threads or processes run on multiple cores. For example, OpenMP or pthreads. Windows can schedule threads across my 4 physical cores and 8 logical processors, so each core runs its own instruction stream on its own data.

So the same CPU can act like SISD, SIMD, or MIMD depending on the workload — it's not stuck in just one category.

Task2

```

1,9.083604
D:\University Courses\pm>python bench.py
Threads N,Run 1,Run 2,Run 3,Avg TN,Speedup SN,Efficiency EN
1,10.099218
1,9.594098
1,8.454488
-> N=1: Avg=9.382601s, Speedup=1.00x, Efficiency=100.0%
2,5.139295
2,4.909750
2,5.220982
-> N=2: Avg=5.090009s, Speedup=1.84x, Efficiency=92.2%
4,3.608492
4,3.689474
4,5.012650
-> N=4: Avg=4.103539s, Speedup=2.29x, Efficiency=57.2%
8,3.643322
8,3.657039
8,3.738415
-> N=8: Avg=3.679592s, Speedup=2.55x, Efficiency=31.9%
16,3.759053
16,3.344824
16,3.624229
-> N=16: Avg=3.576036s, Speedup=2.62x, Efficiency=16.4%
32,3.464201
32,3.355026
32,3.649454
-> N=32: Avg=3.489561s, Speedup=2.69x, Efficiency=8.4%

```

Q2.1: Scaling Saturation & Hardware Contention Analysis

Looking at the numbers, the speedup starts falling apart pretty quickly. N=1 to N=2 still gives a decent 1.84x speedup with 92.2% efficiency, but by N=4 the efficiency drops to 57.2%, and at N=8 it's already down to 31.9%. After that, adding more threads barely helps — N=16 gives 2.62x and N=32 gives 2.69x, which is almost no improvement over N=8.

The main reason is my CPU: it's an i5-1135G7 with 4 physical cores and 8 logical processors. So up to N=4, I'm using real physical cores. But even there, the scaling isn't perfect. Part of that is Turbo Boost — a single core can boost up to around 4.2 GHz, but when all 4 cores are active, the clock drops closer to the 2.4 GHz base. So N=1 is running at a higher frequency than N=4, which makes the speedup look worse than it should.

Once I go past N=4, I'm using Hyper-Threading logical cores. Those share the same execution units and L1/L2 caches, so they don't give anywhere near a full core's worth of performance. At N=8, I'm already seeing heavy contention. By N=16 and N=32, Windows is time-slicing 32 processes onto 8 logical processors, which adds context-switch overhead. On top of that, all threads are fighting for the same 8 MB L3 cache and memory bandwidth.

Python's multiprocessing also has its own overhead — creating processes and passing data between them isn't free, and at high thread counts the work per process gets small enough that this overhead starts to matter.

So for this machine, the practical limit for efficient scaling is around 4 threads. Beyond that, you're just adding overhead and contention without getting much useful work done.

Task3

```
PS D:\University Courses\pm> for ($i=0; $i -lt 10; $i++) { python race.py unlocked }
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=6174.461
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=5519.635
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=6307.850
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=6479.607
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=5993.049
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=5831.514
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=6257.275
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=6329.234
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=5738.641
mode=unlocked output=10000000 expected=10000000 error=0 time_ms=6135.684
PS D:\University Courses\pm> python race_mp.py locked
mode=locked output=10000000 expected=10000000 error=0 time_ms=56865.257
PS D:\University Courses\pm> for ($i=0; $i -lt 10; $i++) { python race_mp.py unlocked }
mode=unlocked output=1248223 expected=10000000 error=8751777 time_ms=3175.305
mode=unlocked output=1290396 expected=10000000 error=8709604 time_ms=3373.333
mode=unlocked output=1523366 expected=10000000 error=8476634 time_ms=3931.969
mode=unlocked output=1331249 expected=10000000 error=8668751 time_ms=3335.134
mode=unlocked output=1323948 expected=10000000 error=8676052 time_ms=3392.232
mode=unlocked output=1330546 expected=10000000 error=8669454 time_ms=3409.376
mode=unlocked output=1256477 expected=10000000 error=8743523 time_ms=3379.091
mode=unlocked output=1358103 expected=10000000 error=8641897 time_ms=4512.797
mode=unlocked output=1472193 expected=10000000 error=8527807 time_ms=5469.026
mode=unlocked output=1405314 expected=10000000 error=8594686 time_ms=6373.404
PS D:\University Courses\pm> |
```

Run	Measured Output	Error (10 ⁷ - Act)	Run	Measured Output	Error (10 ⁷ - Act)
#1	1248223	8751777	#6	1330546	8669454
#2	1290396	8709604	#7	1256477	8743523
#3	1523366	8476634	#8	1358103	8641897
#4	1331249	8668751	#9	1472193	8527807

Run	Measured Output	Error ($10^7 - \text{Act}$)	Run	Measured Output	Error ($10^7 - \text{Act}$)
#5	1323948	8676052	#10	1405314	8594686

Unlocked = 4035.17 ms (10 times)

Locked = 56865.257 ms

Q3.1: Read-Modify-Write (RMW) Window:

At the machine-instruction level, `counter += 1` is not a single atomic operation. It actually breaks down into three steps:

text

Load counter -> register

Add 1 to register

Store register -> counter

When two processes run this simultaneously, they can interleave. For example:

1. Process A loads counter = 100 into its register.
2. Process B also loads counter = 100 into its register.
3. Process A adds 1 and stores 101 back to memory.
4. Process B adds 1 and stores 101 back to memory.

The final value is 101 instead of 102. Process B overwrote Process A's update. This is called a lost update. The cache coherence protocol keeps the cache line consistent, but it does not prevent two cores from reading the same old value before either writes back. This is exactly why we see the huge errors in the unlocked run.

Q3.2: The Synchronization Penalty

From my measurements, the unlocked version averaged around 4035 ms, while the locked version took 56865 ms. The locked version is roughly 14 times slower.

This huge penalty happens because:

- Every increment must acquire and release a mutex. This involves atomic instructions and memory barriers, which are expensive.

- The critical section is serialized. Even though I have 10 processes, only one can increment at a time. The parallelism is effectively destroyed.
- The lock variable and shared counter often sit on the same cache line. Multiple cores competing for that line cause cache-line ping-pong, which adds a lot of overhead.
- Processes may block, sleep, and wake up, which adds context-switch overhead.

In short, the unlocked version is fast because it has zero synchronization, but its result is completely wrong. The locked version buys correctness at the cost of serialization and cache coherence overhead.

Task4

Parallel Fraction Derivation (p): Formula: $p = 2 \times (T1 - T2) / T1$

$T1 = 9.382601 \text{ s}$ $T2 = 5.090009 \text{ s}$

Calculated $p = 0.9150$

Amdahl Theoretical Speedup Ceiling

Formula: $S_{max} = 1 / (1 - p)$

Sequential fraction $(1 - p) = 0.0850$

Max Theoretical Speedup ($N \rightarrow \infty$) = 11.76x

Amdahl 64-Core Projection

Formula: $S_{64} = 1 / [(1 - p) + (p / 64)]$

Projected Speedup on 64 Cores = 10.07x

Gustafson Scaled Speedup (Weak Scaling)

Formula: $S_{gustafson} = (1 - p) + (p \times 64)$

Projected Scaled Speedup (64x Problem) = 58.65x

Q4.1: Strong vs. Weak Scaling Analysis

The difference comes down to what assumption each model makes about the problem size.

Amdahl's Law assumes strong scaling: the total problem size stays fixed. As I add more cores, the parallel part finishes faster, but the sequential part (the 8.5% in my case) takes the same amount of time no matter how many cores I use. Eventually, the sequential part dominates the total runtime. That is why Amdahl's formula predicts a hard ceiling of $1 / (1 - p)$, which for my data is only about 11.76x — even with infinite cores, I can never beat that limit. At 64 cores, the projected speedup is just 10.07x.

Gustafson's Law assumes weak scaling: the problem size grows along with the number of cores. In this model, the sequential part stays constant, but the parallel part grows as you add more cores. So the total time does not stay fixed — it increases, but the parallel portion becomes a larger and larger fraction of the total work. That is why Gustafson's formula gives a scaled speedup of $(1 - p) + p \times N$, which for 64 cores is about 58.65x, much closer to linear.

Neither model is wrong. They just answer different questions. Amdahl's Law tells you what happens if you try to solve the same fixed problem faster. Gustafson's Law tells you what happens if you scale the problem to fit the machine. For my workload, if I keep the problem size fixed, I hit a wall around 11.76x. But if I let the problem grow with the hardware, I can get much closer to linear scaling.